

COM4573

# ARTIFICIAL NEURAL NETWORKS

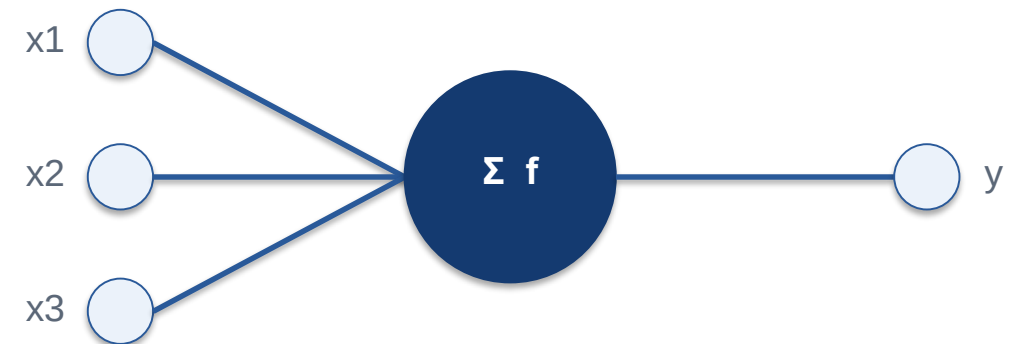
Yapay Sinir Ağları

**Week 1 • Course Introduction and Neural Network Foundations**

Fall Semester 2026-2027

**Instructor: Dr. Hilal Akarkamçı**

Faculty of Engineering • Department of Computer Engineering



# QUESTION OF THE WEEK

Start with intuition before equations.

**Can a machine learn a rule that we never explicitly programmed?**

Think of handwriting recognition, spam filtering, speech recognition, or image classification.

**What does “learning” mean for a computer?**

We will return to this question after building our first artificial neuron.

# Course information

Position of the course in the 2026-2027 curriculum.

|                                                       |                                                   |                                              |                                    |
|-------------------------------------------------------|---------------------------------------------------|----------------------------------------------|------------------------------------|
| <b>Course code</b><br>COM4573                         | <b>Course title</b><br>Artificial Neural Networks | <b>Status</b><br>4th Year Technical Elective | <b>Semester</b><br>7th<br>Semester |
| <b>Weekly hours</b><br>3 theoretical + 0 practice/lab | <b>National credit</b><br>3                       | <b>ECTS</b><br>4                             | <b>Language</b><br>English         |

Official curriculum: Ankara University, Computer Engineering (English), 2026-2027

# Course roadmap

A proposed 15-week learning path from neurons to modern neural networks.

---

**WEEK 1** Introduction & ANN Foundations

**WEEK 2** Perceptron & Linear Classifiers

**WEEK 3** Multilayer Perceptrons

**WEEK 4** Activation & Loss Functions

**WEEK 5** Backpropagation

**WEEK 6** Optimization & Gradient Descent

**WEEK 7** Generalization & Regularization

**WEEK 8** Midterm Review / Assessment

**WEEK 9** Convolutional Neural Networks

**WEEK 10** Sequence Models: RNN & LSTM

**WEEK 11** Attention & Transformer Basics

**WEEK 12** Autoencoders & Representation Learning

**WEEK 13** Training Practice & Model Evaluation

**WEEK 14** Applications, Explainability & Ethics

**WEEK 15** Integration, Case Studies & Final Review

# Course learning goals

What you should be able to do by the end of the semester.

---

- Explain the mathematical and computational foundations of artificial neural networks.
- Design feedforward neural networks for classification and regression problems.
- Derive and interpret forward propagation, loss computation, and backpropagation.
- Select activation functions, optimization strategies, and regularization methods appropriately.
- Evaluate neural network performance and diagnose underfitting, overfitting, and training instability.
- Recognize the core ideas behind CNNs, recurrent models, attention, and representation learning.
- Discuss limitations, interpretability, bias, and responsible use of neural network models.

# Recommended Textbooks

## PRIMARY TEXTBOOK

**Aggarwal, C. C. (2023). Neural Networks and Deep Learning: A Textbook (2nd ed.). Springer.**

DOI: 10.1007/978-3-031-29642-0

## ADDITIONAL REFERENCES

Goodfellow, I., Bengio, Y., & Courville, A. (2016).

**Deep Learning. MIT Press.**

Official online edition: [deeplearningbook.org](https://deeplearningbook.org)

Alpaydın, E. (2020).

**Introduction to Machine Learning (4th ed.).**

MIT Press.

# Why artificial neural networks?

They learn useful representations from examples rather than relying only on hand-written rules.

## PATTERN RECOGNITION

Images, signals, speech, text

## NONLINEARITY

Model complex input-output relationships

## REPRESENTATION LEARNING

Learn useful internal features

## SCALABILITY

Benefit from data and computation

**Neural networks are powerful, but they are not magic. Their behavior follows from data, architecture, objective functions, and optimization.**

# Where do neural networks fit?

A useful conceptual map.

## Artificial Intelligence

### Machine Learning

### Artificial Neural Networks

Deep Learning = neural networks with multiple representation-learning layers

# Biological inspiration, computational abstraction

ANNs are inspired by neurons, but they are mathematical models, not biological replicas.

## Biological neuron

Receives signals through dendrites, integrates them, and transmits activity through an axon.

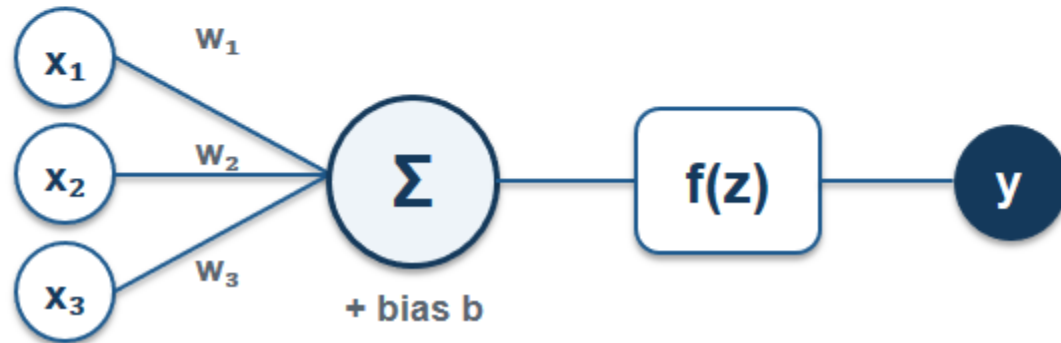
## Artificial neuron

Receives numerical inputs, forms a weighted sum, adds a bias, and applies an activation function.

**In this course, the important question is not “Does it look like the brain?” but “What computation does the model perform, and how does it learn?”**

# Core terms of an artificial neuron

Five terms you should be able to identify before we use the neuron equation.



$$z = w_1x_1 + w_2x_2 + w_3x_3 + b \rightarrow y = f(z)$$

## 1 Inputs (x)

Numerical features presented to the neuron.

## 2 Weights (w)

Learned parameters controlling each input's influence.

## 3 Bias (b)

Learned offset that shifts the neuron's response.

## 4 Weighted sum (z)

Inputs are multiplied by weights, added together, then combined with the bias.

## 5 Activation function f

Transforms  $z$  into the neuron output and can introduce nonlinearity.

## READ THE NEURON FROM LEFT TO RIGHT

inputs  $\rightarrow$  weighted influence  $\rightarrow$  bias  $\rightarrow$  weighted sum  $\rightarrow$  activation  $\rightarrow$  output

# Inputs, weights, bias, activation

Each component has a distinct role.

## Inputs $x$

Measured or encoded features presented to the neuron.

## Weights $w$

Learned parameters that control the influence of each input.

## Bias $b$

A learned offset that shifts the activation threshold.

## Activation $f$

Transforms the weighted sum and introduces useful nonlinearity.

# A neuron

Before training, suppose the weights are already given.

---

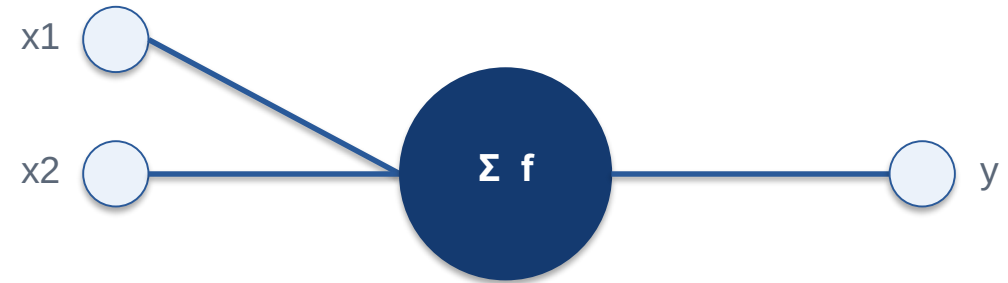
$$x_1 = 1, \quad x_2 = 0.5$$

$$w_1 = 0.8, \quad w_2 = -0.4, \quad b = 0.2$$

$$z = (0.8)(1) + (-0.4)(0.5) + 0.2 = 0.8$$

If  $f(z)$  is a step function with threshold 0:

$$z > 0 \Rightarrow y = 1$$



# What does the activation function do?

It converts the weighted sum  $z$  into the neuron's output  $y$ .

$$z = \sum(w_i x_i) + b \quad \rightarrow \quad y = f(z)$$

**Decision**

The activation function determines how strongly a neuron responds to its input.

**Nonlinearity**

Nonlinear activations allow multilayer networks to learn relationships that a purely linear model cannot represent.

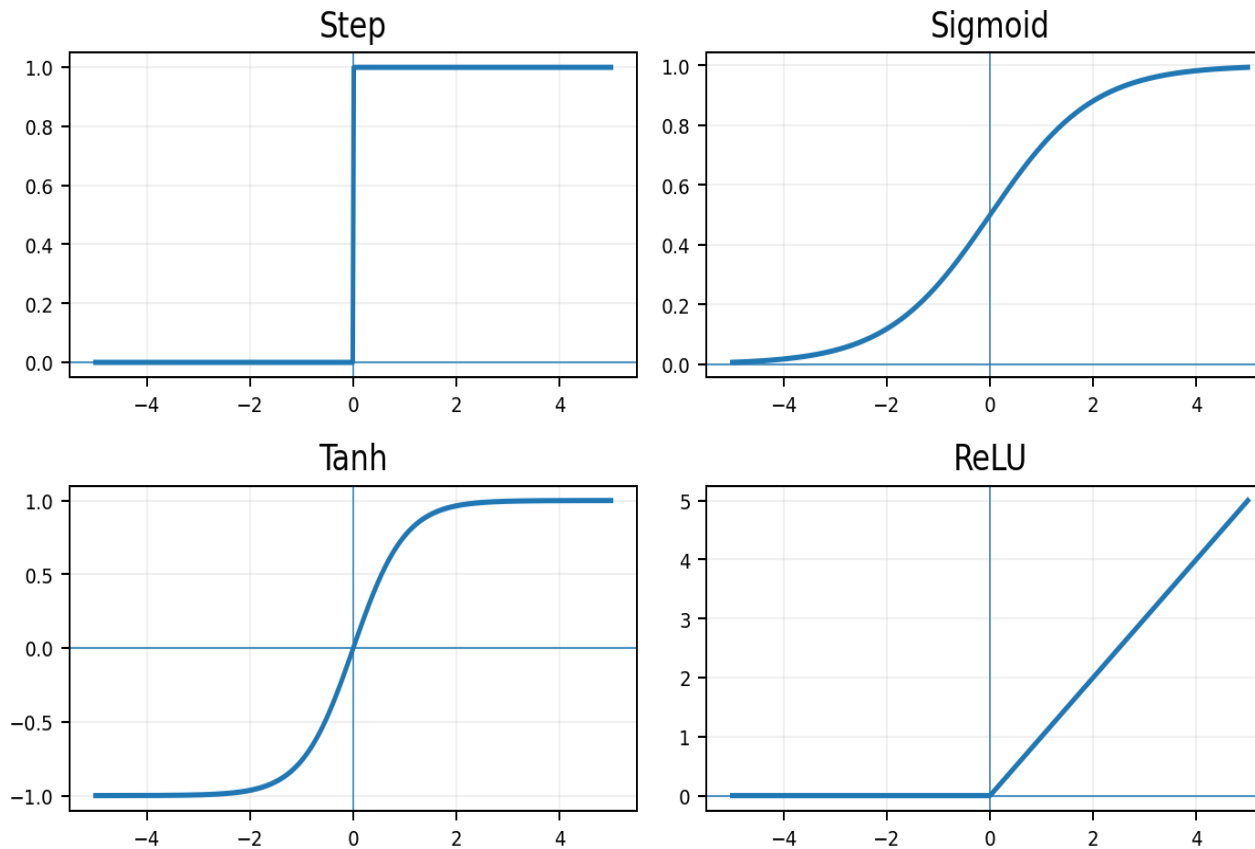
**Output behavior**

The chosen function also affects the range and interpretation of the neuron's output.

Example from the previous slide:  $z = 0.8$ . With a step function and threshold 0,  $f(0.8) = 1$ .

# Common activation functions

Different activation functions are useful for different network components and tasks.



## Step

**0 or 1**

Classic perceptron; useful for explaining threshold decisions.

## Sigmoid

**0 to 1**

Often useful when an output is interpreted as a binary-class probability.

## Tanh

**-1 to 1**

Zero-centered smooth activation; historically common in hidden/recurrent units.

## ReLU -Rectified Linear Unit

**0 to  $\infty$**

A common default for hidden layers in many feedforward and convolutional networks.

Later: activation choice, derivatives, saturation and training behavior.

# From a neuron to the perceptron

A perceptron is a simple trainable linear decision unit.

## Decision rule

Compute a weighted sum and apply a threshold.

## Geometry

The weights and bias define a decision boundary.

## Learning

Adjust parameters when predictions are incorrect.

**Important limitation: a single perceptron can represent only linearly separable decision boundaries.**

This limitation motivates multilayer neural networks.

# What does “learning” mean?

Training is parameter adjustment guided by an objective.



**Training repeats this cycle many times. The learned model is the resulting set of parameters.**

# A supervised learning example

Binary classification: spam or not spam.

## Input

Numerical features derived from an email.

## Target

0 = not spam, 1 = spam

## Prediction

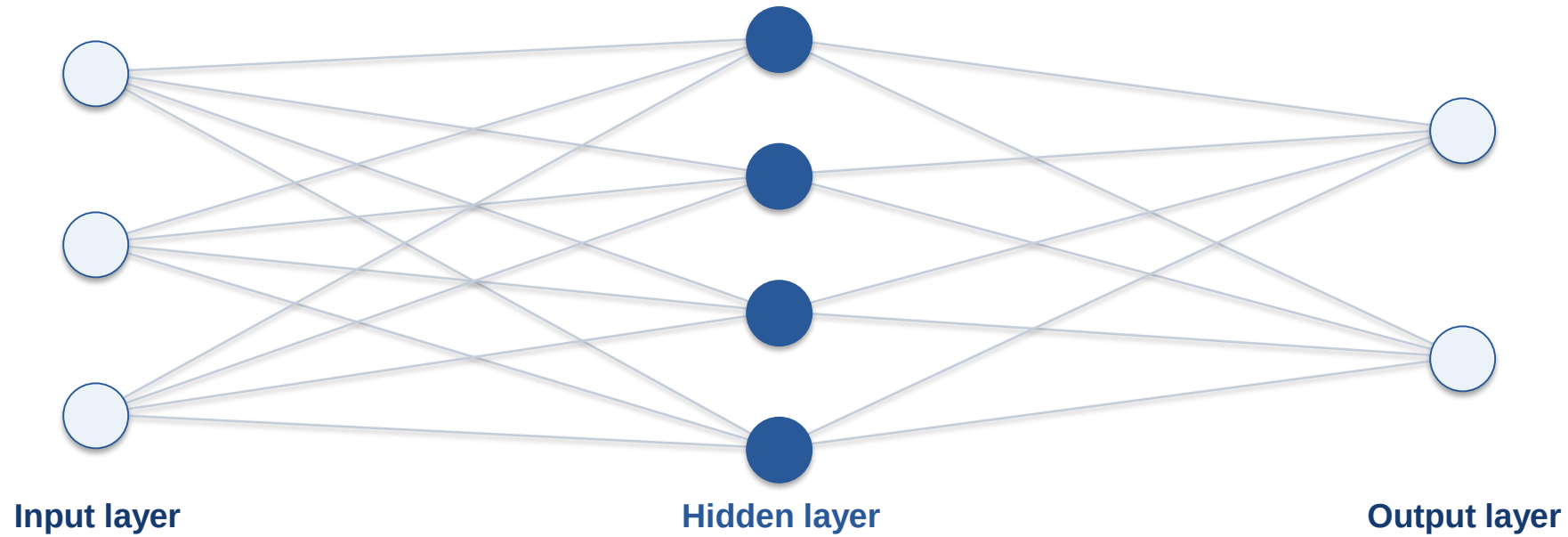
A score or probability for the spam class.

The network does not memorize a verbal rule such as “if the word FREE appears, mark spam.” It learns parameter values that reduce prediction error across training examples.

# From one neuron to a network

Neurons are organized into layers.

---



# Why hidden layers?

They allow the model to build intermediate representations.

## Early representation

Simple combinations of input features.

## Intermediate representation

More useful patterns formed from earlier features.

## Output representation

Evidence organized for the prediction task.

**Depth gives a network multiple stages of transformation. More depth is not automatically better; architecture must match the problem and data.**

# Activation functions • First look

Without nonlinear activations, stacked linear layers collapse into a linear transformation.

## Step

Classic perceptron threshold

## Sigmoid

Smooth output in  $(0, 1)$

## Tanh

Smooth output in  $(-1, 1)$

## ReLU

$\max(0, z)$ , widely used in hidden layers

**We will study their derivatives, saturation behavior, and practical trade-offs in detail.**

# How does a network know it is wrong?

A loss function converts prediction error into a numerical training signal.

---

## Regression

Typical idea: penalize distance between prediction and target.

## Classification

Typical idea: penalize low probability assigned to the correct class.

## Training objective

**Find parameter values that make the average loss on training examples smaller.**

# Learning is not the same as memorizing

A useful model must perform well on unseen data.

## Training set

Used to adjust model parameters.

## Validation set

Used to guide model choices and monitor generalization.

## Test set

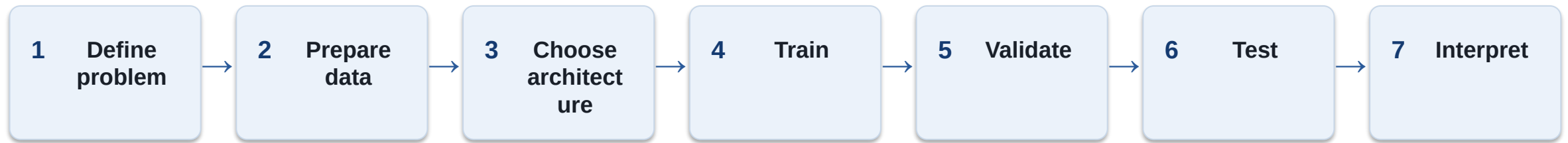
Used for final evaluation on unseen examples.

**A model can achieve very low training error and still fail on new data. This is overfitting.**

# A practical neural network workflow

The course will repeatedly return to this pipeline.

---



**Good neural network practice is more than choosing a model. Data quality, evaluation design, and interpretation matter.**

# Where are neural networks used?

Examples across computer engineering and data-driven systems.

## Computer Vision

classification • detection • segmentation

## Natural Language

classification • translation • generation

## Signals & Time Series

forecasting • anomaly detection •  
sensor analysis

## Cybersecurity

intrusion detection • malware  
classification

## Robotics

perception • control • decision support

## Engineering Systems

fault diagnosis • predictive maintenance

# What neural networks do not guarantee

Powerful models still require careful engineering and critical evaluation.

---

- High accuracy does not automatically imply robustness or fairness.
- More parameters do not automatically produce better generalization.
- A confident prediction can still be wrong.
- Training data can encode noise, imbalance, bias, or leakage.
- Neural networks can be difficult to interpret without additional analysis.
- Compute, energy use, privacy, and deployment constraints may matter.

# THINK • What should the network learn?

Small-group discussion: 3 minutes.

## Scenario

**You want to predict whether a machine component is likely to fail within the next 24 hours.**

- What could the inputs be?
- What should the output represent?
- What examples would you need for training?
- What could make the model fail after deployment?

# Discussion • One possible framing

There is no single correct feature set, but the learning problem must be defined precisely.

---

## Inputs

Temperature, vibration, load, pressure, operating history, recent changes.

## Target

Failure within 24 h: yes/no, or estimated failure risk.

## Training data

Historical measurements paired with reliable failure outcomes.

## Risks

Sensor drift, rare failures, changing operating conditions, leakage, imbalance.

# BACK TO THE QUESTION OF THE WEEK

Can a machine learn a rule that we never explicitly programmed?

---

**Yes, in a specific computational sense.**

We define a model, provide data and an objective, then use an optimization procedure to adjust its parameters. The resulting decision function is learned from examples rather than written as a complete set of explicit rules.

**Learning = parameter adaptation from data under an objective.**

# Week 1 • Key takeaways

What you should leave the first lecture with.

---

- Artificial neural networks are mathematical models composed of connected computational units.
- A neuron combines inputs using weights and bias, then applies an activation function.
- Networks learn by adjusting parameters to reduce a loss function.
- Hidden layers enable intermediate representations and nonlinear decision functions.
- Training performance is not enough; generalization to unseen data is essential.
- Neural networks sit within machine learning and form the foundation of deep learning.

# Before Week 2

Prepare for the perceptron and linear decision boundaries.

## Review

Weighted sum, bias, activation, training cycle.

## Think

When can one straight line separate two classes?

## Preview

Perceptron learning rule and linear separability.

## Week 2: Perceptron • Decision boundaries • Linear separability • Learning rule

# From classroom models to real problems

A neural network project begins with a problem, data, a measurable target, and an evaluation plan.

## 1 • DEFINE THE PROBLEM

What decision or prediction is needed?

Who will use the result?

What would count as a useful outcome?

## 2 • REPRESENT THE DATA

Images • sensor signals • transactions • text • audio • time series

The input representation must match the problem.

## 3 • LEARN AND EVALUATE

Choose a network and loss function.

Train on examples.

Evaluate on unseen data and examine failure cases.

**This section uses project ideas to connect ANN concepts with systems students may actually build.**

# Project 1 • Medical image classification

Can an ANN help identify suspicious patterns in medical images?

## REAL-WORLD PROBLEM

Clinical images contain complex visual patterns. ANN-based computer vision can support image analysis, but a classroom model is not a clinical diagnostic system.

## POSSIBLE INPUT DATA

X-ray, CT, MRI, retinal or other labeled medical images.

Images may require resizing, normalization and augmentation.

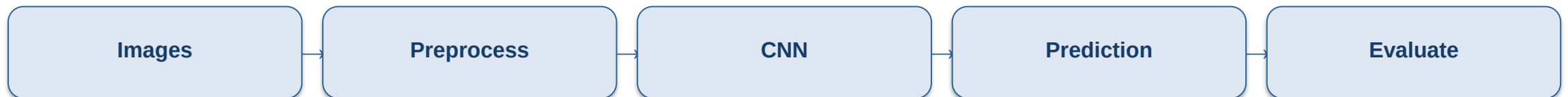
## ANN TASK

Image classification or segmentation.

A CNN can learn spatial patterns directly from pixels.

## MODEL OUTPUT

Class probabilities, a predicted category, or a pixel-level region of interest.



## Project 2 • Predictive maintenance

Can sensor measurements warn us before a machine fails?

### REAL-WORLD PROBLEM

Unexpected equipment failure causes downtime and cost. Sensor histories can contain patterns that appear before failure.

### POSSIBLE INPUT DATA

Vibration • temperature • acoustic signals • pressure • current • operating history.

### ANN TASK

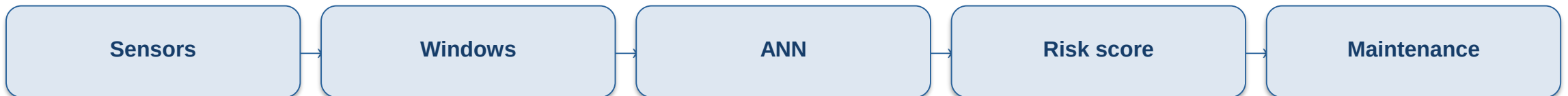
Classification: normal vs. fault type.

Regression: remaining useful life.

Sequence models can use temporal patterns.

### MODEL OUTPUT

Failure probability, fault class, anomaly score, or estimated remaining life.



## Project 3 • Fraud detection

Can a network recognize transactions that do not look legitimate?

### REAL-WORLD PROBLEM

Fraud patterns can be nonlinear, sequential and high-dimensional. A useful system must also manage class imbalance and false alarms.

### POSSIBLE INPUT DATA

Transaction amount • time • merchant • device signals • account history • transaction sequence.

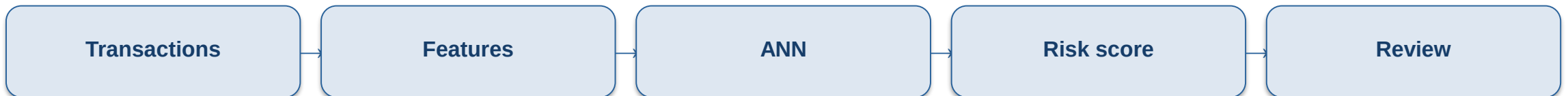
### ANN TASK

Binary classification or sequence classification.

The model learns patterns associated with legitimate and suspicious behavior.

### MODEL OUTPUT

Fraud probability or risk score used to prioritize further checks.



## Project 4 • Visual quality inspection

Can a camera and ANN detect defective products on a production line?

### REAL-WORLD PROBLEM

Manual inspection can be slow and inconsistent at scale. Computer vision can automate repetitive visual checks.

### POSSIBLE INPUT DATA

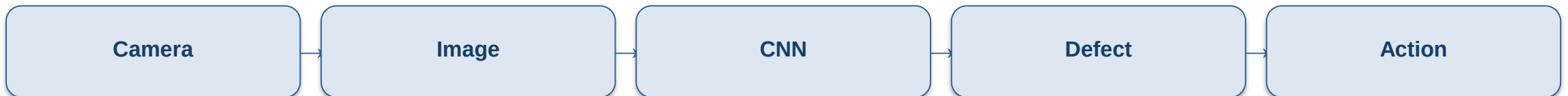
Images or video of manufactured parts, surfaces, packages or assemblies.

### ANN TASK

Image classification, object detection or segmentation using convolutional architectures.

### MODEL OUTPUT

Defect / no defect, defect type, bounding box, or defect mask.



## Project 5 • Demand and energy forecasting

Can past observations help predict what will be needed next?

### REAL-WORLD PROBLEM

Forecasts support capacity planning, inventory decisions and energy management. Temporal leakage must be avoided.

### POSSIBLE INPUT DATA

Historical demand • time • calendar variables • weather • prices • sensor readings.

### ANN TASK

Time-series regression.

MLP, RNN/LSTM or other sequence architectures can model temporal relationships.

### MODEL OUTPUT

Next-hour, next-day or next-week demand estimate with an error measure.

History

Time features

ANN

Forecast

Decision

## Project 6 • Text and sentiment classification

Can a network infer the category or sentiment of a text?

### REAL-WORLD PROBLEM

Text is high-dimensional and context-dependent. Neural representations can learn useful linguistic patterns from examples.

### POSSIBLE INPUT DATA

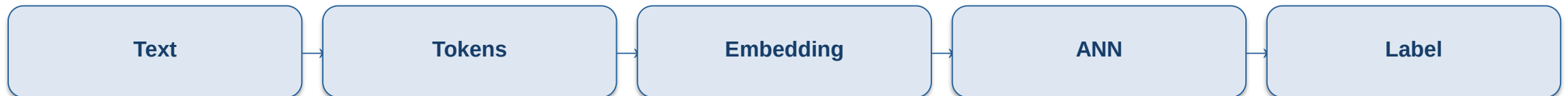
Customer reviews • support messages • news text • survey comments • documents.

### ANN TASK

Text classification using learned embeddings and neural sequence or attention-based models.

### MODEL OUTPUT

Positive / neutral / negative sentiment, topic label, intent class, or document category.



## Project 7 • Environmental anomaly detection

Can sensor networks help flag unusual environmental measurements?

### REAL-WORLD PROBLEM

Monitoring systems produce continuous data. ANN models can help identify complex deviations, but alerts require domain validation.

### POSSIBLE INPUT DATA

Radiation • air quality • temperature • humidity • water-quality or other monitoring time series.

### ANN TASK

Prediction or reconstruction-based anomaly detection; supervised classification when reliable labels exist.

### MODEL OUTPUT

Anomaly score, alert candidate, or predicted environmental state.

Sensors

Time series

ANN

Anomaly

Verify

## Project 8 • Human activity recognition

Can wearable sensors recognize what a person is doing?

### REAL-WORLD PROBLEM

Wearable sensing is a compact example of how raw signals become windows, features or learned representations and then predictions.

### POSSIBLE INPUT DATA

Accelerometer • gyroscope  
• heart-rate or other wearable/mobile sensor windows.

### ANN TASK

Multiclass classification using MLPs, CNNs or sequence networks.

### MODEL OUTPUT

Activity class such as walking, sitting, standing or running, with confidence.

Wearable

Window

ANN

Activity

Evaluate

# Project design challenge

Choose one real-world problem and turn it into a learnable ANN task.

## A • PROBLEM

State the problem in one sentence.

Who benefits from the prediction?

Is it classification, regression, forecasting, detection, segmentation, or anomaly detection?

## B • DATA

What are the inputs?

What is the target?

Where would labels come from?

What train, validation and test split would make sense?

## C • SUCCESS

What metric would you report?

What type of error is most costly?

What risks or limitations should be communicated?

**3 minutes individually → 4 minutes in pairs → brief class discussion**

# Homework • Review and project thinking

10 questions covering the Week 1 concepts and the new real-world project section.

## PURPOSE

Check whether you can explain the basic ANN pipeline, distinguish key components, and formulate a real-world problem as a neural-network learning task.

## INSTRUCTIONS

Answer in your own words.

For design questions, justify your choices.

No coding is required unless you want to explore the problem further.

**Suggested time: 35–45 minutes**

## Homework • Questions 1–5

Foundations and interpretation

1. Explain the difference between artificial intelligence, machine learning, artificial neural networks, and deep learning.
2. For an artificial neuron, explain the roles of inputs, weights, bias, weighted sum, and activation function.
3. In your own words, what does it mean for a neural network to “learn” from data?
4. Why do we normally separate data into training, validation, and test sets?
5. A binary classifier produces a value close to 1 for one sample and close to 0 for another. What might these outputs represent, and why should they not automatically be treated as certainty?

## Homework • Questions 6–10

Real-world ANN project formulation

6. Predictive maintenance: identify at least four possible sensor inputs and define one useful model output.
7. Fraud detection: explain why accuracy alone can be misleading when fraudulent transactions are rare.
8. Visual quality inspection: formulate the task as classification, detection, or segmentation. Explain why you chose that formulation
9. Environmental monitoring: propose an ANN-based anomaly-detection project. Define the input data, output, and one important limitation.
10. Design your own ANN project. State the real-world problem, input data, target, ANN task type, evaluation metric, and one ethical or operational risk.

## Before Week 2

Use the homework to identify what needs another look before we study the perceptron in detail.

- ☐ Can I explain what an artificial neuron computes?
- ☐ Can I distinguish training from evaluation?
- ☐ Can I identify a sensible input and target for a real-world ANN problem?
- ☐ Can I tell whether a task is classification, regression, forecasting, segmentation, or anomaly detection?
- ☐ Can I explain why a high-performing model may still be unsuitable for deployment?

**Week 2 • Perceptron • Linear decision boundaries • Learning rule**

## Sources for the real-world project examples

The project examples are teaching formulations based on established ANN/AI application areas.

- 1 IBM. What Is a Neural Network? Applications include computer vision, NLP, speech, forecasting and pattern recognition.
- 2 IBM. Artificial Intelligence Business Use Cases. Examples include medical imaging, predictive maintenance, manufacturing inspection, finance and energy forecasting.
- 3 IBM. What Is AI Fraud Detection? Neural networks are used with sequential, temporal and high-dimensional fraud signals.
- 4 IBM. Anomaly Detection in Machine Learning. Examples include sensor-based predictive maintenance, medical imaging and fraud/anomaly monitoring.
- 5 Core textbooks : Aggarwal (2023); Goodfellow, Bengio & Courville (2016); Russell & Norvig (4th ed.).

# References & course sources

Core sources for the introductory material.

---

- Ankara University, Faculty of Engineering. Computer Engineering (English) Curriculum, 2026-2027.
- Aggarwal, C. C. (2023). Neural Networks and Deep Learning: A Textbook (2nd ed.). Springer. <https://doi.org/10.1007/978-3-031-29642-0>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Russell, S., & Norvig, P. (2022). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

# Real-World ANN Project Topics

Ten practical projects for applying artificial neural networks to realistic engineering and data-driven problems.

- Choose one clearly defined real-world problem.
- Prepare the data and design an appropriate ANN model.
- Train, validate and test the model using suitable metrics.
- Present the practical output, limitations and failure cases.

# Smart Building Energy Consumption Prediction

Predict next-hour or next-day electricity consumption using historical energy use, time variables and environmental measurements. Students can compare a feedforward MLP with an LSTM-based sequence model.

## ANN TASK

**Regression • MLP • LSTM**

## PRACTICAL OUTPUT

Predicted energy consumption and a graph comparing predicted and actual values.

# Machine Fault Detection from Sensor Data

Use vibration, temperature, current or similar sensor measurements to distinguish normal operation from machine faults. The project connects neural networks with predictive maintenance.

## ANN TASK

**Classification • MLP • 1D-CNN**

## PRACTICAL OUTPUT

A system that reports Normal or Fault Type and provides a prediction score.

# Visual Defect Detection for Manufacturing

Train a convolutional neural network to detect scratches, cracks or other visible product defects. Students work with image preprocessing, augmentation and model evaluation.

## ANN TASK

**Computer Vision • CNN**

## PRACTICAL OUTPUT

Upload a product image and return Defective or Non-defective with confidence.

# Human Activity Recognition from Wearable Sensors

Recognize activities such as walking, sitting, standing or running from accelerometer and gyroscope data collected by wearable or mobile devices.

## ANN TASK

**Multiclass Classification • 1D-CNN • LSTM**

## PRACTICAL OUTPUT

Predict the activity represented by a sensor-data window and report confidence.

# Customer Review Sentiment Classification

Classify customer reviews as positive, negative or neutral. The project introduces text preprocessing, embeddings and neural-network-based text classification.

## ANN TASK

**NLP • Embeddings • Neural Classifier**

## PRACTICAL OUTPUT

Enter a review and display its predicted sentiment and class probability.

# Credit Card Fraud Detection

Estimate whether a transaction is legitimate or suspicious. Students must address class imbalance and evaluate the model using precision, recall, F1-score and a confusion matrix.

## ANN TASK

**Binary Classification • Imbalanced Data**

## PRACTICAL OUTPUT

A fraud-risk score for each transaction and a classification decision.

# Air Quality Level Prediction

Use PM2.5, PM10, temperature, humidity and related environmental measurements to predict an air-quality category or pollutant concentration.

## ANN TASK

**Regression or Classification • MLP**

## PRACTICAL OUTPUT

Predicted air-quality level or a numerical pollution estimate.

# Traffic Flow Prediction

Forecast future traffic intensity using historical traffic observations and temporal variables. Students can compare a basic MLP with an LSTM sequence model.

## ANN TASK

**Time-Series Forecasting • MLP • LSTM**

## PRACTICAL OUTPUT

Next-hour traffic prediction and a graph of actual versus predicted traffic.

# Network Intrusion Detection

Use network-traffic features to distinguish normal connections from attacks. The task can also be extended to recognize multiple types of cyberattacks.

## ANN TASK

**Cybersecurity • Binary or Multiclass Classification**

## PRACTICAL OUTPUT

Normal or Attack prediction, optionally including the predicted attack type.

# Autoencoder-Based Sensor Anomaly Detection

Train an autoencoder using normal sensor behavior and identify unusual observations through reconstruction error. This demonstrates unsupervised or semi-supervised ANN use.

## ANN TASK

**Anomaly Detection • Autoencoder**

## PRACTICAL OUTPUT

An anomaly score and a time-series plot highlighting unusual observations.